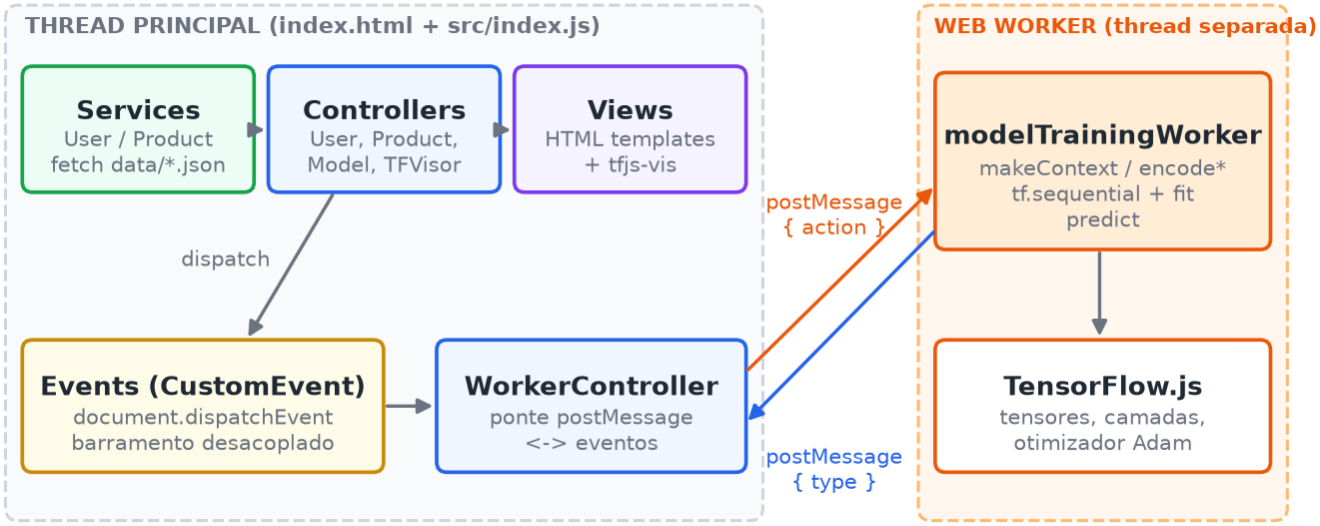
Sistema de Recomendação com TensorFlow.js

Resumo de estudo do projeto *exemplo-01-ecommerce-recomendations*: como ele funciona, da arquitetura ao modelo, e o que fixar. Números e gráficos vêm de um treino real do próprio projeto.

1. O que o projeto faz

Uma loja com 10 produtos e 5 usuários. Uma rede neural aprende, a partir do histórico de compras, a dar uma **nota de 0 a 1** para cada par (usuário, produto): a chance daquele usuário comprar aquele produto. Recomendar = calcular a nota de todos os produtos e ordenar do maior para o menor. Tudo roda no navegador, sem servidor de ML.

2. Arquitetura



O treino roda num Web Worker para a interface não travar. A comunicação é só por mensagens.

- **MVC + Services:** Services buscam dados (`data/users.json`, `products.json`); Controllers decidem; Views só desenham (templates HTML e gráficos do tfjs-vis).
- **Barramento de eventos:** `events.js` usa `CustomEvent` no `document`. Quem dispara não conhece quem escuta. Isso desacopla os módulos.
- **Web Worker:** `new Worker(url, { type: 'module' })`. A página manda `{ action, ...data }`; o worker responde com `{ type, ... }` (`training:log`, `recommend`, `training:complete`). O `WorkerController` traduz essas mensagens em eventos.
- **Constantes de eventos:** `constants.js` centraliza os nomes. Um nome errado não dá erro: o evento só nunca chega. Sempre confira a grafia.

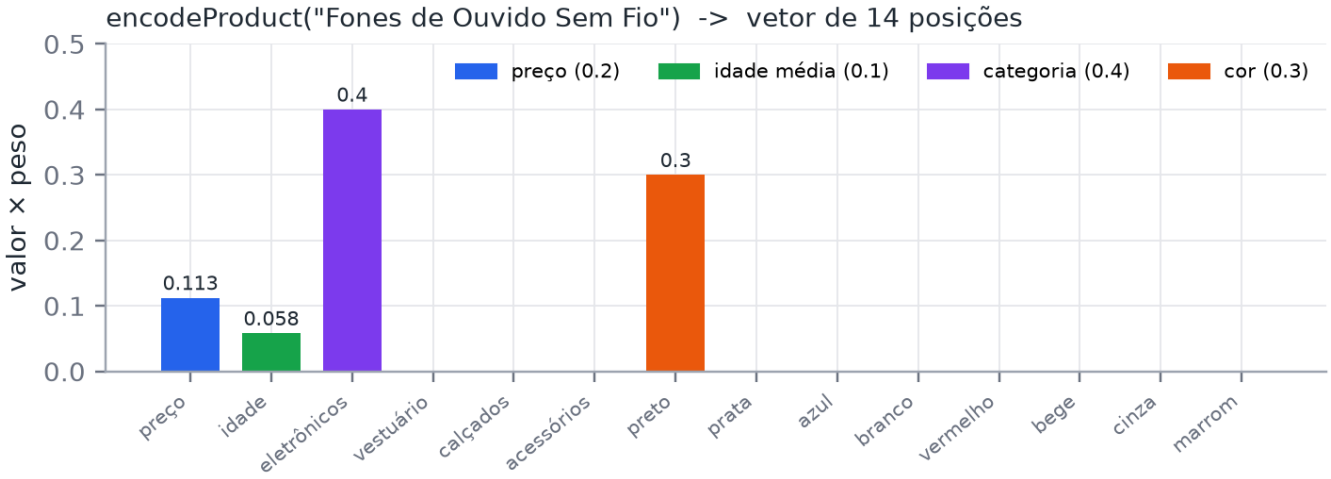
3. Ambiente e depuração (o que deu errado e por quê)

- **Precisa de servidor HTTP.** Abrindo como `file://`, módulos ES, `fetch` dos JSON e o Worker são bloqueados. Use `npm start` (browser-sync, porta 3000).
- **Debug no VS Code:** `.vscode/launch.json` com `"type": "msedge"` + `preLaunchTask` que sobe o servidor. O js-debug entra também no worker.
- `debugger` **só pausa com DevTools aberto** e só se a execução chegar até ele. Um erro antes (ou um erro de sintaxe no arquivo) impede.
- **Erro de sintaxe em módulo = módulo inteiro não roda.** Ex.: `(v, a, b) = (...)` sem o `>` da arrow.
- **Erros em funções async do worker são silenciosos** (Promise rejeitada). Se nada acontece, procure typos: `uesr`, `weigth`, `tf.contatId`.
- **Cache:** após editar o worker, recarregue com `Ctrl+Shift+R`. O `model.fit` pausa se a aba estiver em segundo plano (usa requestAnimationFrame entre lotes).

4. Engenharia de features: transformar coisas em números

Rede neural só entende números. Cada produto vira um vetor; isso é feito em `makeContext` (calcula mínimos, máximos e índices) e `encodeProduct`.

- **Normalização min-max:** `(valor - min) / (max - min)` deixa preço e idade entre 0 e 1. Sem isso, preço (até 199) dominaria categoria (0 ou 1).
- **One-hot:** categoria e cor não têm ordem, então viram um vetor com um único 1. `tf.oneHot(índice, total)`.
- **Pesos:** multiplicam cada parte para dizer o que importa mais: categoria 0.4, cor 0.3, preço 0.2, idade 0.1.
- **Idade média por produto:** média das idades de quem comprou. Personaliza: produto comprado por jovens fica perto de quem é jovem. Sem compras, usa a idade do meio.



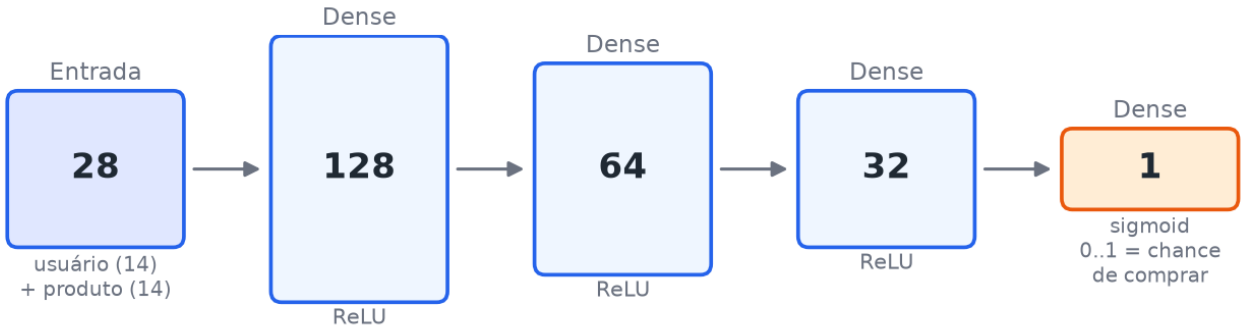
14 = 2 (preço, idade) + 4 categorias + 8 cores. `tf.concat1d([price, age, category, color])` *junta as partes.*

5. Vetor do usuário

- **Com compras:** média dos vetores dos produtos comprados (`tf.stack(...).mean(0)`). O usuário "vive" no mesmo espaço dos produtos que gosta.
- **Sem compras (cold start):** não há histórico, então só a idade é preenchida; o resto é zero. O modelo recomenda pelo que pessoas de idade parecida compraram.

6. Dados de treino e a rede neural

`createTrainingData` cruza **cada usuário com cada produto** (5 × 10 = 50 exemplos). Entrada = [vetor do usuário, vetor do produto] (28 números). Label = 1 se comprou, 0 se não.

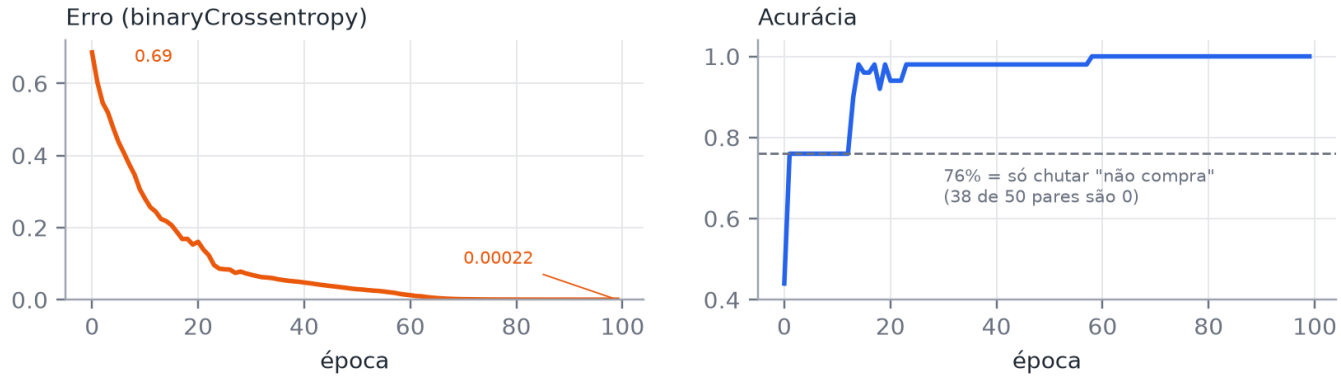


Camadas Dense ligam todos os neurônios. ReLU dá não-linearidade. Sigmoid na saída espreme o resultado entre 0 e 1, virando uma probabilidade.

```
const model = tf.sequential()
model.add(tf.layers.dense({ inputShape: [28], units: 128, activation: 'relu' }))
model.add(tf.layers.dense({ units: 64, activation: 'relu' }))   // inputShape só na 1ª camada
model.add(tf.layers.dense({ units: 32, activation: 'relu' }))
model.add(tf.layers.dense({ units: 1,  activation: 'sigmoid' }))
model.compile({ optimizer: tf.train.adam(0.01), loss: 'binaryCrossentropy', metrics: ['accuracy'] })
```

7. Treinamento

- **binaryCrossentropy:** a função de erro para respostas sim/não. Pune muito quando o modelo tem certeza e erra.
- **Adam (0.01):** o otimizador que ajusta os pesos. A taxa de aprendizado é o tamanho do passo: 0.1 é grande demais e o treino oscila.
- `model.fit(xs, ys, { epochs: 100, batchSize: 32, shuffle: true })`**:** época = uma passada por todos os dados. É assíncrono: use **await**, senão o código segue antes do treino terminar.
- `callbacks.onEpochEnd`**:** a cada época manda `loss` e `acc` via `postMessage`; a View desenha a curva com tfjs-vis.

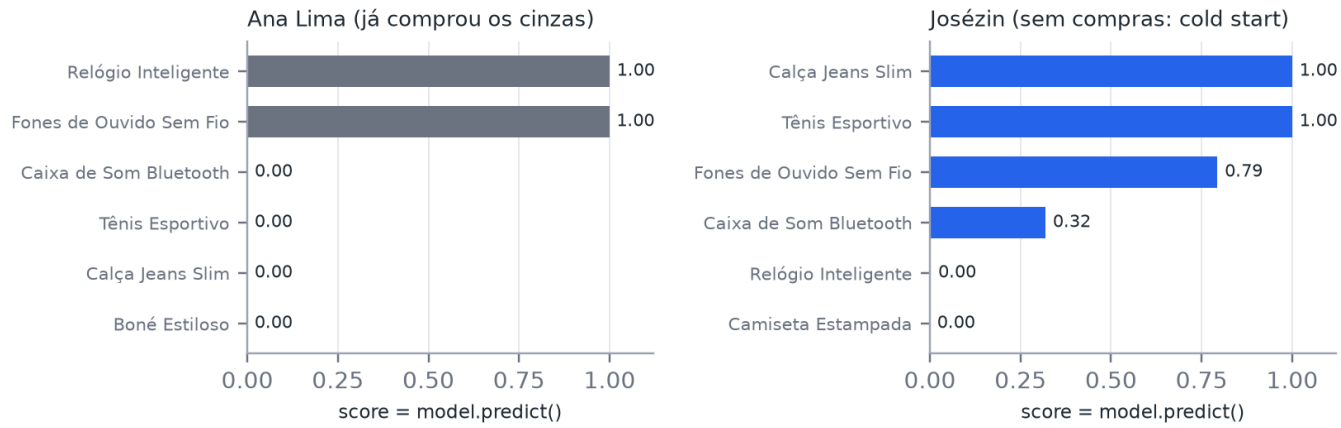


Treino real: o erro cai de forma contínua; a acurácia fica parada em 76% e depois sobe para 100%.

Lição: acurácia engana com dados desbalanceados. Só 12 de 50 pares são compras. Um modelo que sempre responde "não compra" já acerta 76%, por isso o platô no começo. Olhe também o erro. E 100% aqui significa que o modelo **decorou** 50 exemplos, não que generaliza.

8. Recomendação

`recommend` monta [usuário, produto] para os 10 produtos, chama `model.predict(tf.tensor2d(inputs))`, lê as notas com `dataSync()`, ordena e envia.



Barras cinzas = produtos que o usuário já comprou. Na prática, filtre-os antes de mostrar.

9. Para fixar

- Pipeline de ML: **dados -> features (números) -> rede -> treino (fit) -> previsão (predict)**.
- Normalize valores contínuos; use one-hot para categorias; pesos dão prioridade às features.
- Recomendação como classificação binária: "este usuário compra este produto?" e ordenar pela probabilidade.
- Trabalho pesado vai para o Web Worker; a comunicação é por `postMessage` e eventos com nomes centralizados.
- Tensores ocupam memória: em projetos maiores use `tf.tidy()` ou `.dispose()`.